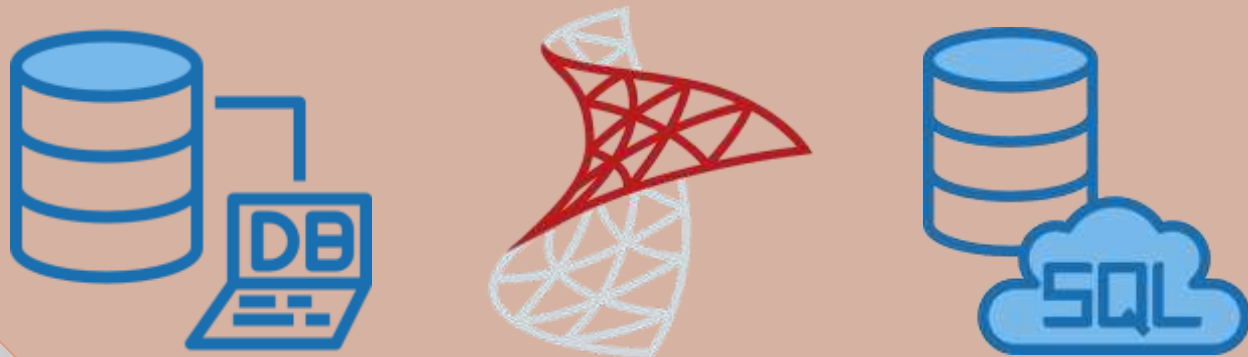


Database (Lecture #2)



Dr. Syed Saqib Jamal
Assistant Professor (조교수)

College of IT & Convergence
(Department of Computer
Engineering)





Lecture Content

주차	기 간	수 업 내 용 및 학 습 활 동	비 고
1	09/01 ~ 09/07	Course introduction, What is a database? Why databases are used? Overview of Database systems	Database, DBMS
2	09/08 ~ 09/14	Types of databases, Relational database basics, DBMS Architecture	RDBMS, Tables

Today's Agenda

Types of Databases

Relational Database Basics

DBMS Architecture

RDBMS

Tables

Types of Databases

Relational Database

NoSQL Database

Hierarchical Database

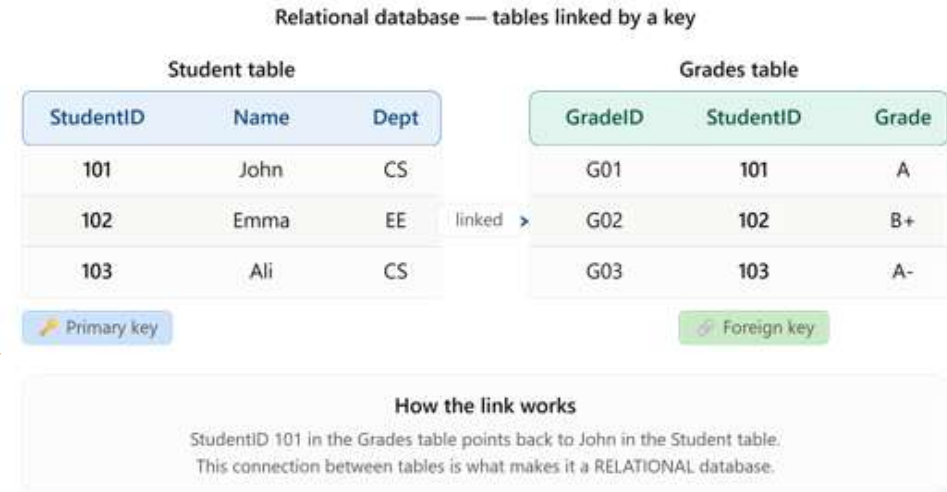
Network Database

Cloud Database

Object-Oriented Database

Relational Database

- Data is organized in **tables**.
- Each **table** with rows and columns.
- Tables are **linked to each other** through keys.
- Uses **SQL** to manage data.
- Example: *"Student table linked to Grades table"*.
- Real life: **MySQL, PostgreSQL, Oracle**



Relational Database - Example

Example — University System:

1. **Student Table** stores student info
2. **Grades Table** stores marks
3. Both tables are **linked** by Student ID

Row (Record/Tuple)

- Each **row** represents one complete entry
- Example: *"Row 1 is all information about John"*
- Also called a **record** or **tuple**

Column (Field/Attribute)

- Each **column** represents one type of information
- Example: *"The Name column stores only names"*
- Also called a **field** or **attribute**

Relational database — tables linked by a key



How the link works

StudentID 101 in the Grades table points back to John in the Student table.
This connection between tables is what makes it a RELATIONAL database.

SQL query example

```
SELECT Student.Name, Grades.Grade
FROM Student
JOIN Grades ON Student.StudentID = Grades.StudentID;
```

→ fetches John → Grade A

Relational Database - Example

Example — University System:

1. **Student Table** stores student info
2. **Grades Table** stores marks
3. Both tables are **linked** by Student ID

Primary Key

- A column that **uniquely identifies** each row
- No two rows can have the same primary key
- Example: "StudentID 101 belongs to only John — no other student can have 101"

Foreign Key

- A column in one table that **links to the primary key** of another table
- Creates a **relationship** between two tables
- Example:

Relational database — tables linked by a key



How the link works

StudentID 101 in the Grades table points back to John
This connection between tables is what makes it a REL

Grades Table:

GradeID	StudentID	Subject	Grade
1	101	Math	A
2	102	Physics	B

Student Table:

StudentID	Name
101	John
102	Emma

→ fetches John → Grade A

Relational Database - Example



Relationships Between Tables

Three types — keep it simple for now:

- **One to One** → One student has one ID card
- **One to Many** → One student has many grades
- **Many to Many** → Many students take many courses

NoSQL Database

- **Not organized** fixed tables — more flexible structure.
- Stores data in **3 ways**:
 - **Documents** → like a JSON file with all info together
 - **Key-Value** → like a dictionary — name: "John"
 - **Graphs** → shows relationships between things
- Perfect for **big and unstructured data**
- Instagram Example:
 - Instead of separate tables for photos, likes, comments
 - Everything about **one post stored together** as one document
- *"When data is too big and too varied for tables — use NoSQL"*
- Real life: **MongoDB used by Instagram, Twitter**



- Used by: Instagram, Twitter, Netflix, Uber

NoSQL Database - Example

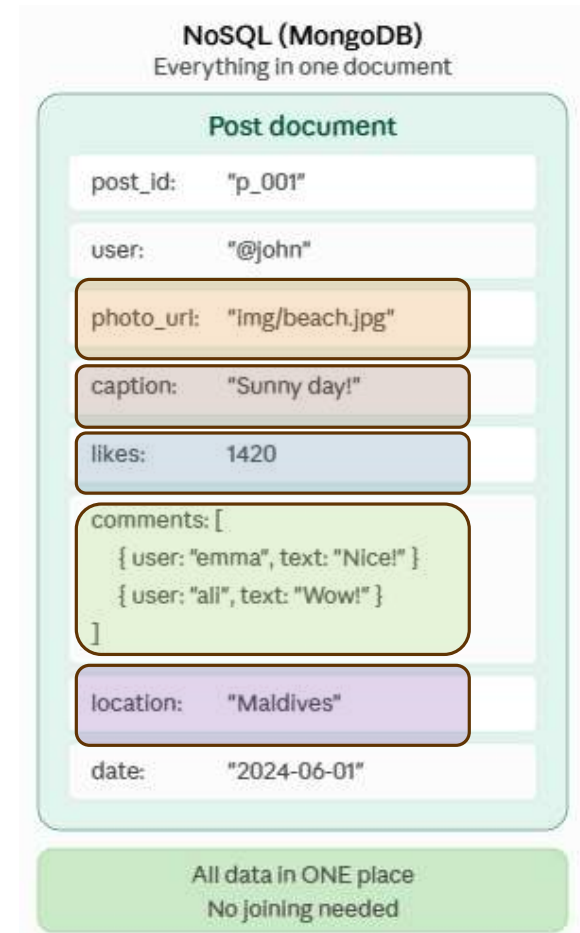


Example - Instagram

"Instead of a table, Instagram stores each post as a document with photo, likes, comments all together"

One Instagram post is stored as a single document.

Everything the photo, caption, likes, comments, location — all sits together in one place. To get a post, you fetch one document and you have everything instantly.



NoSQL is better when data is complex, nested, or needs to be fetched all at once — like social media posts

NoSQL Database-Full Picture



Step 1 — When a user posts on Instagram, all data is saved together as one document — photo, caption, likes, comments all in one place.

Step 2 — That document goes into a collection — think of it as a folder holding all Instagram posts.

Step 3 — When someone likes the post, the database simply updates the likes number inside the same document — no need to touch any other table.

Step 4 — Comments are added as a nested list inside the same document — this is something a regular table simply cannot do.

Step 5 — When someone opens a post, the database fetches just one document and returns everything instantly — no joining 3 separate tables like SQL would need to.

Step 6 — NoSQL spreads data across many servers worldwide, so Instagram can handle billions of posts without slowing down.

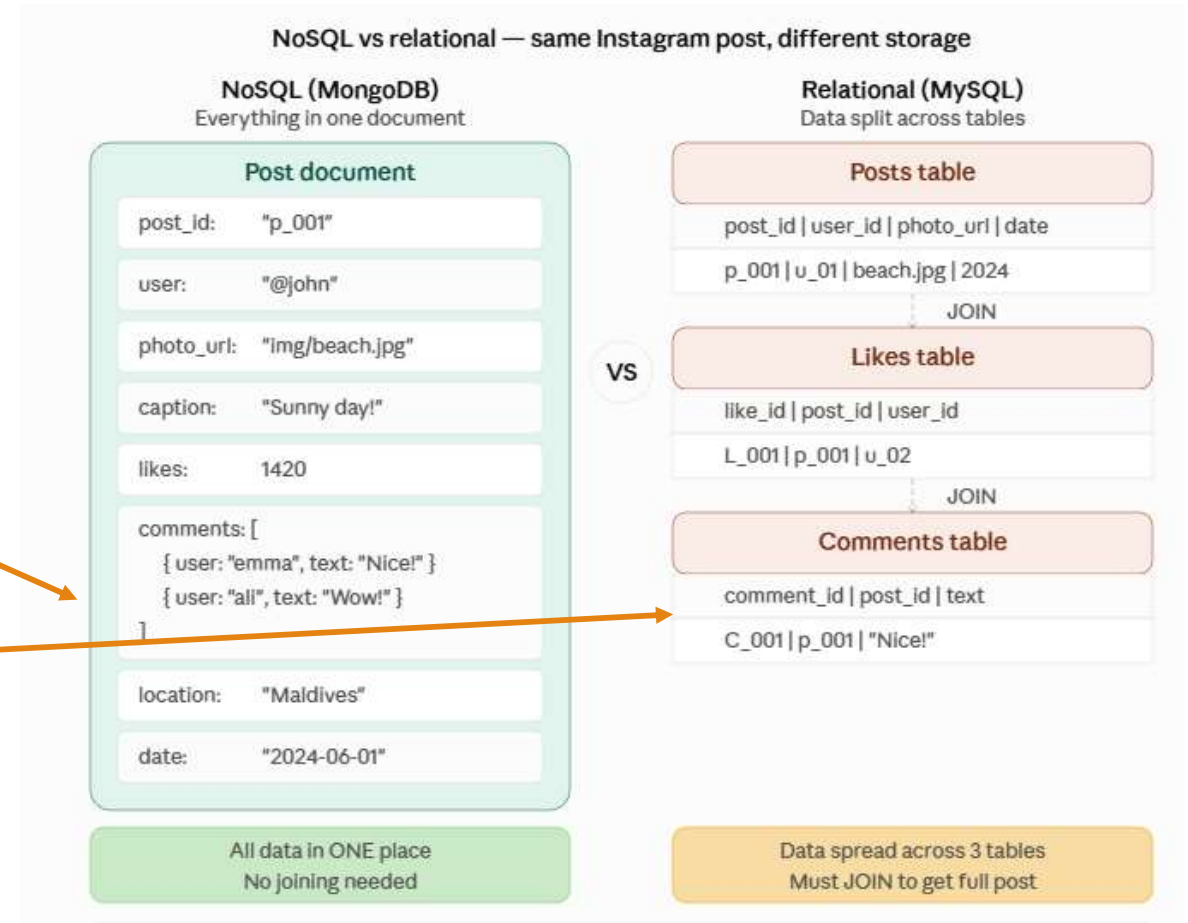


NoSQL vs relational

Same Instagram post, different storage

Left side — NoSQL (MongoDB): One Instagram post is stored as a single document. Everything — the photo, caption, likes, comments, location — all sits together in one place. To get a post, you fetch one document and you have everything instantly.

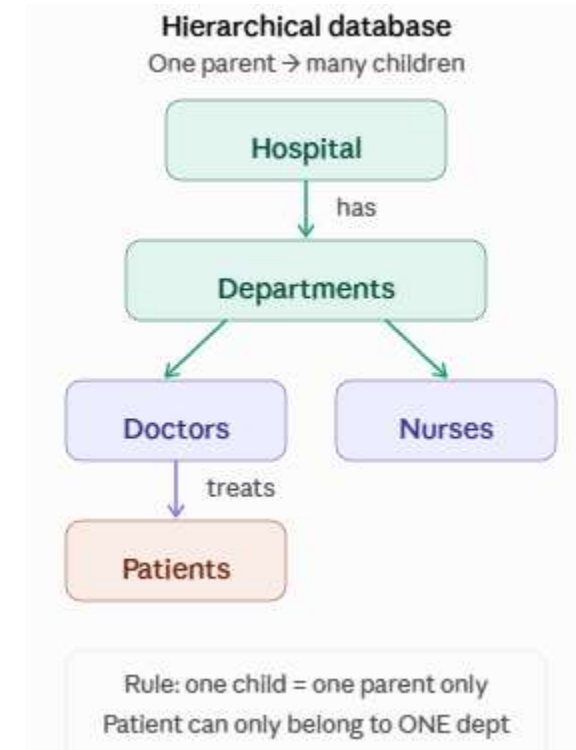
Right side — Relational (MySQL): The same post would be split across three separate tables — Posts, Likes, and Comments. To get the full post, you have to JOIN all three tables together, which is more complex and slower for this type of data.



Hierarchical Database

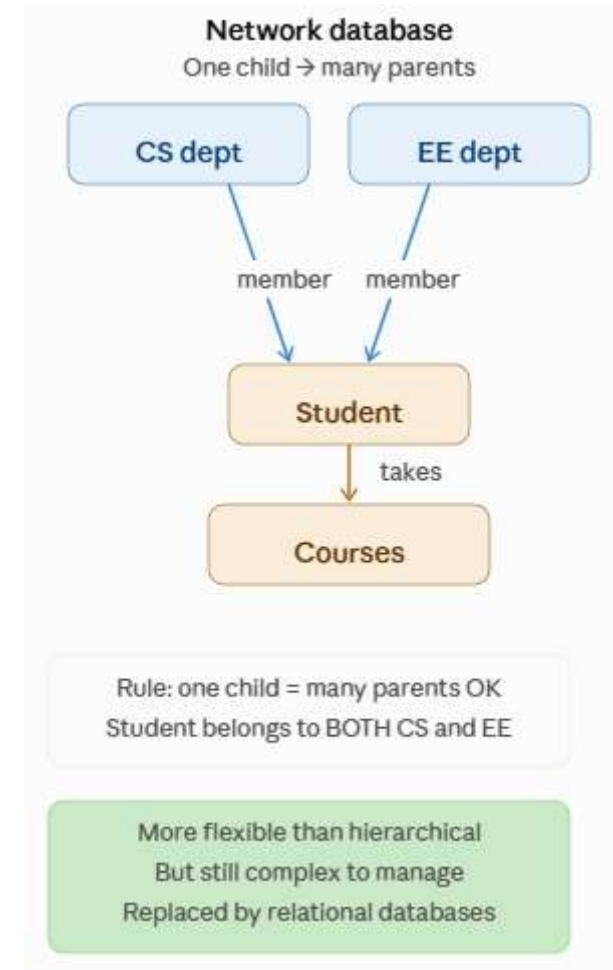


- Data arranged like a **family tree** — top to bottom.
- One **parent** can have many **children**.
- But one child can only have **one parent**.
- Very **fast** for simple lookups going top to bottom.
- Problem: **not flexible** — what if a patient belongs to two departments?
- *"Old style — invented in the 1960s — rarely used today"*
Used by: Old banking systems, IBM mainframes



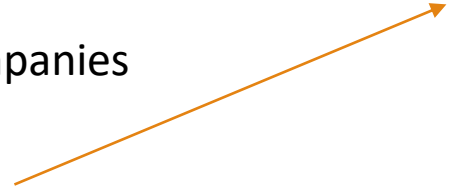
Network Database

- Similar to hierarchical but **more flexible**.
- One child can have **multiple parents**.
- Data is connected like a **web or network**.
- Same student belongs to **both departments** — not possible in hierarchical.
- Still complex and hard to manage.
- *"Also old style — replaced by relational databases"*
Used by: Old telecom systems, legacy enterprise systems



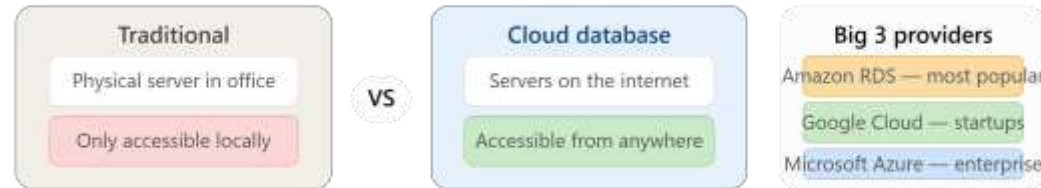
Cloud Database

- A normal database but hosted **on the internet** instead of local servers.
- Access your data from **anywhere in the world**.
- Three big providers:
 - **Amazon RDS** → most popular cloud database service
 - **Google Cloud** → used by many startups
 - **Microsoft Azure** → popular in enterprise companies
- Netflix Example:
 - *"Netflix doesn't own thousands of servers in every country"*
 - *"They store everything on cloud — and stream to you from nearest cloud location"*

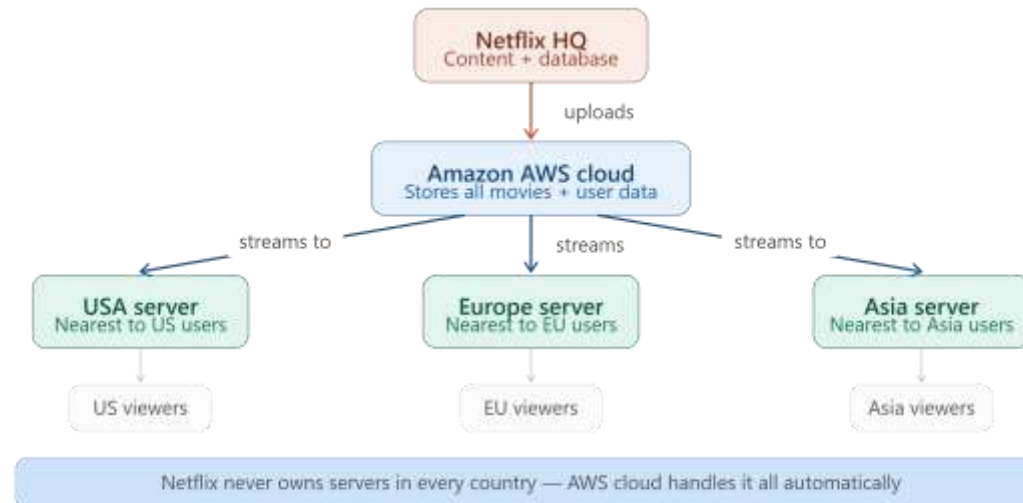


Cloud Database - Example

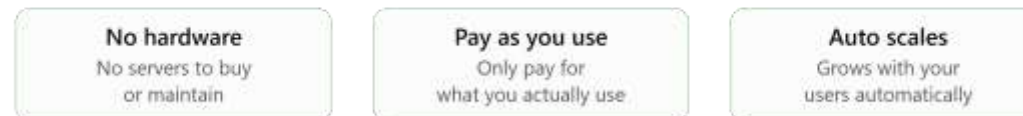
Traditional server vs cloud database



Netflix cloud architecture — real example



Key benefits of cloud databases



IMPORTANT ANNOUNCEMENT...!

- Next Class (Wednesday).
- Complete Lecture will be Uploaded on Tuesday until 12pm.
- The Assignment and Presentation groups needs to be formed and announced after the next class.
- Kahoot Session

Next Class

- Object-Oriented Database
- DBMS Architecture



Thank You

Name: Dr. Syed Saqib Jamal

Email: syedsaqib1609@gachon.ac.kr